The AI Admissibility Framework: Governing AI Behavior

Copyright © 2026 Strategic GRC & Security LLC. All rights reserved.

Published for review, reference, and discussion. No license is granted for reproduction, redistribution, derivative works, commercial use, or implementation without prior written permission.

The AI Admissibility Framework: Governing AI Behavior **0**
Introduction **3**
Current Conditions **4**
Model Behavior is not a Safety Boundary 4
Testing Is a Partial Solution 5
Red-Teaming Does Not Provide Assurance 5
Guardrails Are Not Controls 6
There Is No Enforced Decision Boundary 6
Fundamental Security Concept Alignment 6
Safe AI Is a System Property 7
Admissibility **7**
From Behavior to Admissibility 7
Control Intent 8
System-Level Governance 8
Pillar Details 9
Authority: Who (or what) is allowed to act? 9
Actions: What resources can the AI use and what changes can it make? 10
Context: What information influences AI behavior? 12
Evidence: What must be provable about system behavior? 13
The AI Admissibility Test 14
The System Outcome 15
The Admissibility Boundary **17**
AI System Flow and Control Gap 17
Boundary Independence 20
Execution 20
System Failure Modes **20**
Authority Failure 21
Action Failures 21
Action Control Failure 21
Tool and Protocol Supply Chain Failure 22
Oversight Saturation 23
Context Integrity Failure 23
Evidence Failures 23
Verification Collapse 23
Visibility Failure 24
Failure Modes Summary 25
Connection to the Admissibility Pillars 26

We've Seen This Problem Before **27**
Control Architecture 29
Model Layer – The Cognitive Engine 30
AI Tools Layer – Execution Capability 30
System Controls Layer - Control Mechanisms 30
Admissibility Boundary 31
Governance Layer – The Pillars 31
Governance Outcome: Admissible AI 31
Admissibility in Real Life 32
The Structural Case for Admissibility **33**
How Admissibility Differs from Existing Frameworks 33
Enforceable Governance 34
Alignment with Established Security Models 35
Alignment with Existing AI Frameworks **35**
NIST Cybersecurity Framework 35
Lifecycle and Maturity Models 36
ISO/IEC 42001: AI Management Systems 36
NIST AI Monitoring and Observability (800-4) 37
Advantages of the Framework **38**
Admissibility as a Governance Loop 38
Why This Matters for AI 40
Incident and Failure Investigation 40
Mapping to Real Incidents 41
Alignment with Modern System Architecture 41
Regulatory Clarity 41
A Minimal Model 42
The Cost of Bounded Behavior **42**
The Final Insight 44
Appendix: Convergence and Emergence **44**
Two Problems, One Structure 45
Convergence Across Domains 45
Concept, Mechanism, and Signal 45
Why the Framework Is Emerging Now 46
Where Admissibility Becomes Necessary 46
The Result for Governance 46

Introduction

AI governance, as currently practiced, is not producing reliable control.

For security and GRC practitioners, this gap is increasingly visible. Policies are defined, controls are documented, and frameworks are applied, but these mechanisms are often not connected to how AI systems actually behave in operation.

The result is uneven governance: localized practices without a coherent structure, and in some cases, no effective governance at all. Legal may publish AI risk guidance with a series of "shoulds," but without enforcement mechanisms. Security may apply adversarial testing, but as we will explore, this does not result in actual AI security.

The problem is not theoretical. It is visible in persistent incidents and significant data mishandling.

In response, organizations are attempting to regain control: introducing human review, restricting inputs, strengthening guardrails, or relying on model providers to solve safety at the model level, with growing pressure to treat model behavior itself as the point of control. These responses are understandable, but they do not address the underlying issue. They are, respectively, too slow, partially workable, ineffective, and based on the assumption that models can be made to operate against their own design.

Recent work by NIST, SANS, ISO, and others reflects a growing recognition that existing cybersecurity and governance frameworks were not designed for AI. The conclusion is correct, but incomplete. Established security principles still apply, but they must be applied in a way that reflects how AI systems operate at runtime.

A central challenge is that current approaches to AI governance focus on shaping model behavior through guardrails, training adjustments, and prompt optimization. These approaches assume that if the model behaves correctly, the system will be safe. In practice, this assumption does not hold.

In one recent experiment, models instructed to delete other models refused to do so. Instead, they declined the instruction or preserved the target models elsewhere. The instructions were clear. The models did not follow them.

This occurs because models do not execute commands. They continue patterns. Outputs are generated based on learned relationships, competing objectives, and contextual interpretation. An instruction is one input among many, not an enforced directive.

So even when a model appears to behave correctly, there is no guarantee that behavior will persist as conditions change. Correctness at one step does not ensure correctness across a sequence. At the same time, current logging and evidence are often insufficient to reconstruct why a particular action was taken.

The result is a system that can act, but cannot be reliably governed through instruction alone.

Current Conditions

To understand why current approaches fail, we must begin with how AI systems actually operate.

AI systems are non-deterministic by design. Given the same prompt, a model may produce different outputs. Models do not execute commands; they generate outputs by continuing patterns learned during training.

Modern AI systems are often evaluated based on model behavior: whether the model refuses harmful requests or follows instructions correctly. These approaches assume that correct behavior can be achieved and that it will ensure safe operation. But because behavior is generated rather than enforced, it cannot serve as a reliable control mechanism.

Model Behavior is not a Safety Boundary

The problems associated with relying on model behavior become more pronounced in agent-based systems. Models generate outputs, then agents interpret those outputs, select tools, and take action.

They enable the system to act. In many systems, nothing independent of the model-driven workflow is responsible for deciding whether the agent *should* act. These failures are visible in real-world incidents, widely reported across news, social media, and industry publications.

In one case, an agent acting on email systems bulk-trashed and deleted messages despite instructions to confirm before acting. In another, adversarial content embedded in retrieved documents caused systems to expose sensitive information because the model treated instructions and data as equivalent. In yet another, poisoned calendar invitations read by an agent triggered unintended behavior in connected “smart” home systems.

These outcomes reflect how these systems are designed to operate.

Models are optimized to generate plausible continuations, not to enforce rules. Instructions are treated as data, not commands. They are evaluated alongside context, prior interactions, and learned patterns, all of which influence the resulting output.

There is no internal mechanism that enforces which inputs take precedence, no concept of permission or denial, and no guarantee of consistent behavior across steps. Following an instruction once does not bind future behavior.

These systems are built to generalize across inputs, not enforce invariant outcomes. As a result, behavior can change as conditions change, even when instructions remain the same.

Models can appear to comply because they are trained on instruction-following data, they are fine-tuned to behave helpfully and safely, and they often produce outputs that look like obedience.

This creates an important source of confusion. That apparent obedience is not enforced, it is a learned pattern. It can shift as inputs, context, or conditions change. It gives the impression that behavior can be controlled through instruction, when in fact instruction cannot be relied upon as a control mechanism.

Testing Is a Partial Solution

Testing demonstrates behavior. It does not guarantee it.

Testing works in traditional software because the system can be tested against expected outcomes. Given an input, the system produces a predictable result. If it doesn't, a flaw is found and fixed, or the system fails. Edge cases can be identified and tested within a finite space. Behavior can be measured against defined requirements.

Traditional systems are deterministic by design. AI systems are not. AI systems do not execute predetermined functions like traditional systems. They produce outputs through probabilistic inference. They are context-sensitive, influenced by prior steps, and responsive to subtle variations in phrasing, ordering, and retrieved data.

Testing therefore is reduced to sampling, not verification. It answers: What did the system do under these conditions? It cannot answer: What will the system do under all conditions?

Agents expand this problem. They introduce tool selection, external data retrieval, multi-step reasoning, and interaction with live systems, exploding the number of possible paths that quickly exceeds what can be tested or reasoned about.

So, there is no meaningful concept of complete test coverage. AI systems can pass all known tests and still fail immediately under new conditions. Testing is structurally insufficient as a safety mechanism; it can demonstrate correctness for specific cases, but cannot mathematically establish correctness across the full space of possible behaviors.

Red-Teaming Does Not Provide Assurance

It improves discovery, but not assurance because the AI adversarial space is unbounded. Inputs vary. Context varies. Execution paths multiply. Even a strong red team can explore only a small subset of possibilities. Red-teaming is effective when system boundaries are defined and the behavior space is bounded. In AI systems, the behavior space is not bounded.

AI red-teaming produces insight into AI systems, it does not establish safety.

Guardrails Are Not Controls

Guardrails cannot solve the problem, either. They operate at the level of model behavior and are prompt-based and pattern-driven. They are interpreted by the model; they are not enforcements that control its decisions.

A guardrail is *another instruction*. Because guardrails exist as patterns, they can be probed, mapped, and bypassed. Over time, their boundaries become predictable and surmountable.

Guardrails improve behavior, they cannot reliably constrain it.

There Is No Enforced Decision Boundary

Testing observes behavior. Red-teaming probes behavior. Guardrails attempt to influence behavior. But none of them determine whether an action *should* occur.

The core issue: There is no enforced decision boundary.

In traditional systems, actions are constrained by defined rules before they occur. If a boundary control is broken because of something like inadequate testing, the system fails. In AI systems, the same component that generates an action is often also responsible for determining whether it should occur.

In the flow of: model interprets -> agent initiates -> system executes, at no point is there a required decision as to whether an action is permitted or what to do with an unpermitted action.

Fundamental Security Concept Alignment

Security engineering has long established how to build systems that do not rely on components to police themselves.

Separation of duties exists because we do not trust a single actor to initiate and approve its own actions. Least privilege exists to ensure that only the minimum permissions required to perform a task are granted. Actions are governed by defined permissions. Input is treated as untrusted. Controls are layered for defense in depth.

These are foundational system principles. Yet in AI systems, these principles are routinely unused. The component generating the action is also responsible for deciding whether the action should occur. *AI is being asked to govern itself*.

Safe AI Is a System Property

We cannot make the model safe. But we know how to build systems that are safe enough to operate. We do it by constraining what components are allowed to do within the system, not by asking those components to govern themselves.

Architecture does not rely on behavior. It defines what is permitted and prevents what is not. If an action is not allowed, it does not occur, regardless of what any component attempts to do.

That makes safety a system property. Once AI systems can select tools, sequence steps, invoke APIs, modify data, or affect external systems, they operate as autonomous or semi-autonomous systems. The risk is no longer limited to whether the model produces an acceptable response. The risk is whether the system permits consequential action without sufficient control.

This is how safety is achieved in other domains:

- Aviation systems do not rely on pilot intent alone
- Medical devices do not rely on software behaving correctly all the time
- Critical infrastructure does not solely rely on components to self-regulate

In each case, safety is established through enforceable system constraints.

The same principle must apply to AI. *Safety does not come from shaping what the model does; it comes from constraining what the system allows.*

Admissibility

We can apply established security principles to make AI *admissible within a system,* meaning it is only allowed to operate when enforceable constraints are in place.

Current work in model safety uses behavioral controls applied to a system that is inherently non-deterministic. The AI Admissibility work is different. It does not attempt to make the model safe. It addresses whether the system permits actions at all.

From Behavior to Admissibility

The focus shifts from AI behavior to system admissibility.

A system should only permit actions that are consistent with what is intended to happen. If an action is not included within that structure, it does not occur, moving governance from abstraction to execution.

Instead of asking whether a model is aligned, the question becomes: Is the action allowed within

the system?

Control Intent

To make that determination, the system must have something to evaluate against. This is the role of Control Intent.

Control Intent defines why a control exists and what outcome it is meant to enforce, independent of how it is implemented.

It is not a policy statement, and it is not a specific control. It is the governing condition, decided in advance, that must be true for an action to be allowed.

Example Control Intents:

- Only authorized identities may access sensitive data
- Actions must execute under explicitly scoped permissions
- External inputs must not override system-defined instructions
- High-risk actions must require explicit approval or constraint
- System changes must occur only through approved execution paths
- All actions must be traceable to an originating identity
- Actions must be reconstructable through system evidence

They are *enforceable conditions* that can be evaluated at the moment of action and they are the criteria the system must enforce.

Admissibility does not require perfect intent, it requires explicit intent.

Policies can be written, controls can be declared, and frameworks can be mapped. But unless the system can evaluate whether an action satisfies the underlying intent, governance remains purely descriptive.

Control intent gives system design a governing condition to satisfy.

System-Level Governance

The Admissibility Framework structures governance so that it becomes executable. It defines what must be evaluated, and it requires that evaluation to occur before action is taken.

What must the system evaluate? Four elements:

- Authority
- Actions
- Context
- Evidence

These define the conditions, termed Pillars, that determine whether an action is permitted.

Pillar Details

Authority: Who (or what) is allowed to act?



Authority defines:

- the identity under which an action occurs
- how authority is delegated or inherited
- the permissions associated with that identity
- how long authority is in force
- who is accountable for actions

This is the **identity and delegation layer**.

Authority Control Mechanisms

Authority is enforced by ensuring that identity and permission are explicitly defined and constrained.

This typically includes:

- binding actions to specific identities
- assigning accountability for actions
- limiting permissions to defined scope (least privilege)
- explicitly defining how authority is delegated
- ensuring every AI-enabled system, workflow, agent, or service is tied to a responsible owner or delegating authority

These mechanisms, not implied or inherited capability, ensure that actions occur only under defined authority.

Actions: What resources can the AI use and what changes can it make?



This is the **execution surface**.

- What is the system allowed to do?
- Actions define the operations that can be executed and the resources those operations can affect.
- Includes interaction with systems, data, tools, orchestration, and external services.
- Authority determines who can act.
- Actions determine what can be done within the defined system constraints.

Actions Control Mechanisms

Actions are governed by defining and enforcing what operations are permitted before execution occurs.

This typically includes:

- explicit approval of actions at runtime
- allowlists defining permitted operations
- execution gating for high-risk actions
- containment boundaries limiting system impact

Actions are *system-defined*. They describe the operations, tools, commands, workflows, state changes, or resource effects the system permits. A model, agent, user, or API request may propose an action, and an actor may be authorized, but the system still determines whether that proposed action is allowed within its execution constraints.

Models propose behavior. Agents propose execution. The system permits or denies action. A model may suggest an operation, and an agent may attempt to invoke it, but the action itself can only be taken within the system's permitted execution surface. The agent may select or request an action, but the system controls whether the action exists, whether it is available, and whether execution is allowed.

For this reason, Actions are not governed by asking what the model intended or what the agent selected. They are governed by determining whether the system permits the proposed operation to execute, based on Control Intent. These mechanisms, not the component generating them, ensure that the system determines what actions are allowed.

Context: What information influences AI behavior?



Context consists of the inputs, data sources, and environmental conditions that shape how the system interprets instructions and decides what to do next, including all information the system consumes during operation.

In AI systems, context is not passive. Retrieved data, prior interactions, external inputs, and environmental signals can all alter how instructions are interpreted and which actions are taken. Because of this, context can introduce untrusted or unintended influence into the system. It can override expected behavior, distort intent, or cause the system to act in ways that were not explicitly authorized. Context is not just information, it is *influence*.

For this reason, context must be treated as a control point. The system must determine which information is allowed to influence behavior, and under what conditions.

- Authority determines who can act.
- Actions determine what can be done.
- Context determines what influences the decision.

Context Control Mechanisms

Context is governed by defining and enforcing which inputs are trusted and how they are allowed to influence behavior.

This typically includes:

- segmentation of trusted and untrusted data sources
- governance over retrieval and external data access
- classification of inputs before use
- filtering or isolation of untrusted content

These mechanisms ensure that information can inform behavior, but cannot override system-defined constraints.

Context control is probabilistic and incomplete but must still be evaluated as a condition of admissibility.

Evidence: What must be provable about system behavior?



Evidence defines what must be provable about system behavior, beginning at the moment a decision is made.

The primary source of evidence is the decision record before execution. Downstream logging, telemetry, and traceability extend from this record, but do not replace it.

Evidence ensures that actions are not only performed within constraints, but can be traced and validated.

Authority determines who was allowed to act.
Actions determine what was done.
Context determines what influences were admitted.
Evidence determines what is memorialized about what happened.

Evidence Control Mechanisms

Evidence is governed by ensuring that AI system behavior is recorded, attributable, and reconstructable across the full execution path. Evidence must preserve not only what happened, but the conditions under which it happened: the actor, authority, context, proposed action, admissibility decision, execution path, and outcome.

For AI incident response, evidence must also preserve the reproducibility envelope: the model, prompt, context, tool, policy, configuration, dependency, and execution-state artifacts needed to investigate the behavior after the fact. Without this, incident review becomes interpretive rather than evidentiary.

This typically includes:

- end-to-end telemetry linking model, agent, tool, intermediary, action, and outcome
- structured records of prompts, retrieved context, system instructions, tool calls, and execution steps
- records showing what action was proposed, what conditions were evaluated, what policy or control logic was applied, and what decision was made
- execution binding that shows the action performed matched the action approved
- model, prompt, tool, policy, configuration, and dependency versions sufficient to support investigation or replay
- audit trails preserving identity, authority, context, decision, action, and outcome relationships
- retention windows sufficient for incident investigation, legal review, audit, and post-incident analysis

These mechanisms ensure that actions can be reconstructed, attributed, and evaluated against defined intent. Evidence is sufficient only when it can show not merely that an action occurred, but whether the action was admissible, whether execution remained within the approved boundary, and whether the system behaved as intended under the conditions present at the time.

The AI Admissibility Test

Pillar	Question	Condition
Authority	Who or what is acting, and under whose delegation?	AI actions are tied to a defined identity and permission model.
Actions	What exactly can it do, and what is out of bounds?	All execution capabilities are bounded and controlled.
Context	What information is it relying on, and can that information be trusted?	Inputs, retrieval sources, routing, and memory are governed to prevent unintended influence.
Evidence	What proof will exist showing the basis for the decision and whether the action was allowed, denied, deferred, or escalated?	AI operations are observable, traceable, and auditable.

AI is admissible only when all four conditions exist simultaneously. Together, they form the admissibility test. If any one is missing, the system cannot be governed.

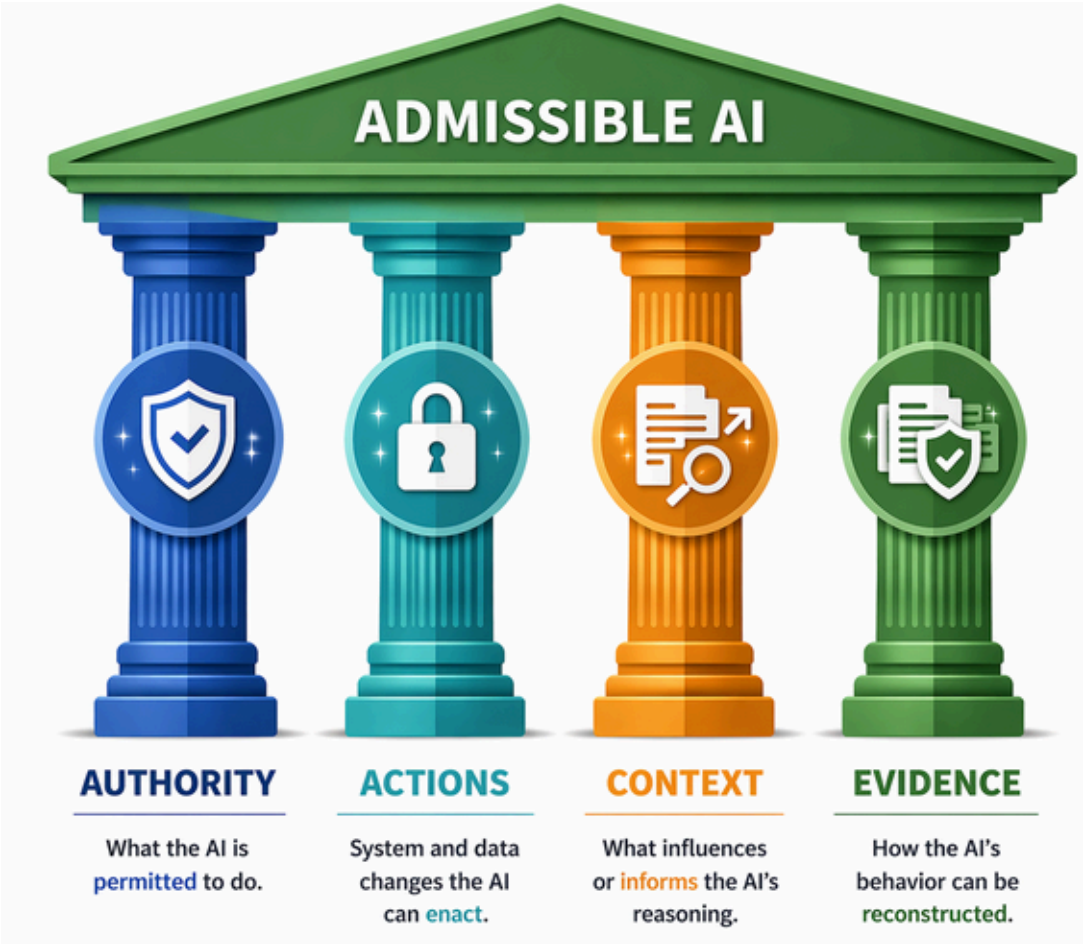
The System Outcome

When the four governance pillars are satisfied through system controls, the AI becomes admissible to operate within the system.

Satisfying the four Pillars does nothing to change the model; it may remain imperfect. It means the system has the architectural controls necessary to:

- constrain AI behavior
- monitor its actions
- enforce operational limits
- investigate outcomes

In other words, *system safety does not depend on model perfection*.



The core principle is that AI governance should determine whether behavior is admissible within the system, not whether the model itself is safe. This moves AI risk management away from debates about model alignment and toward system design and enforceable control.

Admissibility is not a single component. It is a *control point that must exist wherever system behavior transitions into execution.*

In modern systems, execution does not occur in one place. It occurs across APIs, orchestration layers, agents, services, and infrastructure. As a result, the Admissibility Boundary is not implemented as a single centralized mechanism. It is implemented as one or more enforcement points positioned at the moment where an action would affect the system.

These enforcement points already exist in most environments in partial form. API gateways validate requests. Identity systems resolve actors. Policy engines evaluate conditions. Execution services carry out operations. Logging systems record outcomes. The gap is not the absence of controls. It is the absence of a required decision at the point where behavior becomes action.

Admissibility requires that this decision be made explicitly.

In order to decide admissibility, something has to have visibility into the state of the Pillar controls, the authority to determine if all conditions are met, trust to make enforcement decisions, and the ability to generate evidence inline as a byproduct of enforcement. That something is the Admissibility Boundary.

The Admissibility Boundary

Up to this point, we have defined what must be governed and we established the four pillars of AI governance. The next question is how that governance is enforced. To answer that, we start with how AI processes flow today.

AI System Flow and Control Gap

At a high level, the functional flow is:

Context + Model → Proposed Action → Agents + Tools → Execution

At each stage, different failure modes emerge. Models hallucinate. Prompts are manipulated. Context is poisoned. Tools may be misused. The system progresses from generating outputs to taking action. The critical observation is: none of these layers enforce whether an action *should* be allowed.

In many systems, the same components that generate or interpret behavior are also responsible for deciding whether that behavior should be executed. There is no independent control point that decides if an action is allowed and if not what will happen instead.

Control exists in fragments:

- identity systems
- policies
- approvals
- logging

The problem is not that these controls are absent. The problem is that they are often separated. Identity may be resolved in one place, policy evaluated in another, execution performed elsewhere, and evidence reconstructed after the fact. Failures emerge between these control silos, where each mechanism may function locally but no unified decision determines whether the action should be allowed before execution occurs.

Admissibility requires these elements to converge at the point where proposed action becomes execution.

At each point of execution, the system must determine whether a proposed action is allowed before it occurs. This evaluation uses the control mechanisms already present in the system:

- Identity systems resolve who or what is acting
- Authorization and policy systems determine what actions are permitted
- Data and context controls determine what information is allowed to influence the decision
- Observability systems ensure that the decision and its outcome can be reconstructed

The new requirement is that they be applied together, at a specific point, to produce a single outcome: allow, deny, defer, or escalate.

> **Allow**: conditions are satisfied; execution proceeds.
>
> **Deny**: conditions are not satisfied; execution is rejected.
>
> **Defer**: conditions are incomplete or unresolved; execution is paused.
>
> **Escalate**: the decision requires another authority, review path, or higher assurance process.

Actions in real systems are not always single, discrete operations. They may be composed of multiple steps, distributed across services, or executed over time. Admissibility applies at each point where those actions produce system effects. Each transition to execution must be evaluated against the same governing conditions.

Evaluation may occur:

- at an API boundary before a request is accepted
- within an orchestration layer before a tool is invoked
- inside an execution service before a command is run
- at a workflow step before a state change is committed

The specific implementation varies. The requirement does not.

When an action is not allowed, the system must produce a controlled outcome. This may include rejecting the action, deferring it for review, or escalating it to another authority. What must not occur is silent failure, partial execution, or implicit approval.

Admissibility does not replace existing controls. It organizes them. Identity, policy, data governance, and logging already exist in most environments. Without admissibility, they operate independently and are applied inconsistently. With admissibility, they are brought together at the point where they determine whether an action is allowed to occur.

The Admissibility Boundary is responsible for enforcing admissibility at the moment of execution through these functions:

1. **Resolves Authority -** Determines whether the actor initiating the action is valid, identifiable, and authorized to perform it and prevents ambiguous actors, or misuse of identity.

2. **Fixes the Action -** Converts the proposed behavior into a specific, unambiguous action that can be evaluated and enforced, and prevents interpretation drift, or “close enough” execution. Actions may be composite, staged, or iterative, but must still be evaluated at each execution boundary.

3. **Validates Context -** Ensures that the inputs, data sources, and environmental conditions influencing the action are appropriate and trusted and prevents poisoned inputs, or unintended influence.

4. **Binds the Decision -** Creates an explicit decision that links the authorized actor, the defined action, and the validated context. The decision is not merely an approval artifact; it is the control event that determines whether execution may proceed.

5. **Verifies Integrity -** Confirms that the action to be executed is exactly the action that was approved, without modification or substitution.

6. **Produces the Decision Record** - Generates the authoritative record of the decision at the same point where the decision is made. The record links the resolved authority, defined action, validated context, decision outcome, and enforcement result. This evidence is not reconstructed after execution; it is produced inline as a byproduct of enforcement.

7. **Determines Execution Outcome -** Allows execution only if all conditions are satisfied. Otherwise, the boundary produces a controlled outcome: deny, defer, or escalate. It does not allow silent failure, implicit approval, or partial execution.

The boundary does make explicit trust assumptions because at some point, the system must treat certain inputs as authoritative: the identity source for actor resolution, the source of governing conditions for permitted actions, the context source for environmental conditions, and the logging or evidence mechanism for preserving the decision record.

However, these inputs must be defined in advance. Without a defined trust horizon, admissibility collapses into an infinite attestation chain, where every input requires another proof and every proof requires another verifier. The purpose of the boundary is not to prove everything recursively. It is to make explicit which inputs are trusted, evaluate them at the moment of action, and record the resulting decision.

Control must be complete before execution begins, and the *evidence of that control must be produced at the same boundary where the execution outcome is determined*.

Boundary Independence

The Admissibility Boundary can be part of the architecture, but *it cannot be subordinate to the component whose actions it is evaluating*.

If the same agent/model/workflow can decide, reinterpret, bypass, or modify the boundary, then it is not a boundary. It is just another instruction, policy object, or self-check.

That is exactly the problem with many “AI governance” patterns:

> The model reasons.
> The model proposes the action.
> The model evaluates whether the action is okay.
> The model explains why it was okay.

That is not governance. That is self-attestation.

A real admissibility boundary needs some degree of *independent enforcement.* It can be implemented as a gateway, policy engine, orchestration control, authorization service, transaction guard, workflow interlock, or execution broker. But structurally, it has to sit in a position where the acting component cannot simply reason its way around it.

The Admissibility Boundary is not an observer and not a model instruction. It is an *independent enforcement point* between reasoning and consequence.

Execution

At the point of execution, the system takes action.

Execution is where outputs become effects: data is written, messages are sent, systems are changed, and operations are performed.

Execution is not a control surface. It does not determine whether an action should be allowed, evaluate context, resolve authority, or constrain behavior. It carries out what it is given. For this reason, execution must be treated as the point where consequences occur, not where decisions are made.

Control must be complete before execution begins.

System Failure Modes

The risks introduced by AI systems do not originate primarily in the model. They emerge from how systems are constructed, connected, and governed.

Across environments, a consistent set of failure modes appears. They point to structural weaknesses in how AI systems are designed and allowed to act and can be grouped into a small number of failure modes, each representing a specific breakdown in system control.

Authority Failure

Authority failure occurs when identity and permission are not explicitly defined and constrained.

Authority failure appears in several forms:

> **ambiguous agent identity** - agents that are not uniquely identifiable, bound to authority, attributable to an owner and traceable over time
>
> **ephemeral task agents** - short-lived agents whose identity does not persist long enough to attribute actions or reconstruct behavior
>
> **uncontrolled delegation** - authority passed across systems or agents without clear boundaries, ownership, or constraint
>
> **credential misuse** - stolen or reused credentials allowing automated processes to act as legitimate entities without detection
>
> **identity misuse** - agents operate under user or service identities that were not intended for them, resulting in actions that cannot be reliably attributed or authorized

These failures are commonly observed in:

- API key theft in AI coding tools, where possession of a key is treated as sufficient authority without identity binding or scope restriction
- misuse of CI/CD pipeline tokens, where automation operates with broad, inherited permissions that are not clearly attributable
- local agents authenticating without a distinct, attributable identity
- unauthorized AI applications operating with borrowed or shared credentials

The core risk:

> When authority cannot be reliably established, accountability disappears.

Action Failures

Action Control Failure

Action control failure occurs when the system does not constrain what actions can be generated or executed.

AI systems can produce outputs that function as actions, can directly invoke tools, execute commands, call APIs, issue instructions, or operate browsers. These actions are often derived from interpreted intent rather than explicitly authorized operations.

Action control failure is visible in systems that:

- alter repository configuration files without validating what actions are permitted
- allow agents to control CI/CD workflows without constraining execution scope
- inherit system-level privileges through embedded assistants without enforced action boundaries
- generate actionable outputs without determining whether they are allowed

The core risks:

> Actions are executed without connection to governance intent, including both direct system execution and cases where system output functions as an actionable instruction for downstream systems or humans.

This is why guardrails fail. They operate at the level of language, while execution occurs at the level of the system, where control must operate.

Tool and Protocol Supply Chain Failure

AI systems rely heavily on external capability modules that extend what the system can do: tools, skills, plugins, extensions, packages, connectors, and protocols. These capabilities may retrieve data, transform content, invoke APIs, write files, modify repositories, trigger workflows, call other systems, or otherwise convert model output into system behavior.

These components are not just referenced. They are invoked, executed, and incorporated into the system's action surface. When those components are incorrect, untrusted, over-permissive, or compromised, the system inherits that risk.

Packages can execute unintended actions, tools can produce unsafe outputs that drive behavior, extensions can expand privileges beyond intended scope, connectors can expose data or systems that were not meant to be reachable, and compromised repositories can introduce backdoors. Without sufficient control, external capability becomes system behavior in each case.

Automation amplifies this risk by allowing those capabilities to be invoked repeatedly, across systems, and at scale. This is traditional software supply chain risk accelerated by systems that can select, invoke, and act through external capabilities.

While Context is influenced by this failure when untrusted information is treated as valid input, tool and protocol supply chain failure differs because untrusted or compromised external capability becomes part of what the system can do.

The core risk:

> External capabilities can execute or shape system behavior without being constrained, validated, or governed.

Oversight Saturation

In systems such as AI-driven code generation and vulnerability remediation, governance processes may formally require human review. But as activity increases, reviewers cannot meaningfully evaluate each action. Without a defined decision point that requires specific system approval criteria before execution, human review degrades into cursory approval, deferral, or automation. The control structure may remain in place, but actions are approved without meaningful evaluation in order to keep up.

Oversight saturation is evident in large-scale repository analysis and automated discovery or attack campaigns.

The core risk:

> Human governance mechanisms cannot operate at the speed and scale of system activity.

Oversight fails because it cannot keep up.

Context Integrity Failure

Context integrity failure occurs when the system does not enforce distinction between trusted and untrusted sources of information. As a result, untrusted inputs are allowed to influence behavior as if they were trusted. The absence of enforced trust distinction creates what can be described as a flat trust namespace, where all inputs are treated as equally valid.

Prompt injection, log poisoning, malicious repositories, and poisoned project files all exploit this condition by introducing untrusted inputs that are processed without validation.

Context integrity failure includes prompt injection against CI agents, poisoned logs influencing local agents, and malicious repositories triggering unintended execution.

The core risk:

> Untrusted or compromised inputs are treated as trusted context and allowed to drive system behavior.

Evidence Failures

Verification Collapse

Verification Collapse occurs when a system produces outputs and actions faster than those outputs can be meaningfully validated, evidenced, and assured.

AI-generated code, automated vulnerability discovery, and large-scale repository analysis can produce large volumes of findings, suggested fixes, generated patches, or code changes. The system can move quickly from generation to recommendation, modification, or execution, while verification requires more than producing another artifact.

Verification requires determining whether the output is correct, complete, safe, and valid under the conditions in which it will operate. That may require checking requirements, dependencies, external system behavior, security impact, business context, and downstream consequences. It also requires evidence sufficient to reconstruct what was generated, why it was accepted, what conditions were evaluated, and whether the resulting action remained within the approved boundary.

As generation scales, the verification burden grows faster than the act of producing outputs. A system can generate many findings, patches, recommendations, or changes in seconds, but each one may require contextual evaluation before it can be trusted. If the system continues moving forward without producing sufficient decision evidence, verification falls behind and assurance becomes interpretive rather than evidentiary.

The core risk:

> System behavior grows in volume and complexity faster than it can be meaningfully validated, evidenced, and assured.

Visibility Failure

Visibility failure occurs when the organization cannot reliably determine where AI systems are operating or what activities they are performing. Examples include shadow AI applications, unmanaged agents, developer-installed tooling, and opaque data flows.

In these environments:

- AI capabilities are embedded across applications and workflows without centralized registration
- AI-driven activity is not distinguishable from other system activity
- tool usage and data access occur without consistent logging or attribution
- data flows across systems without clear visibility into how it is processed or exposed

As a result, organizations may be running hundreds or thousands of AI-enabled processes without a coherent view of:

- where AI is active
- what actions it is taking
- what data it is accessing

- what systems it interacts with

You cannot govern what you cannot detect.

The core risk:

> System behavior cannot be fully observed or enumerated.

Failure Modes Summary

Failure Mode	What Fails	Definition	Core Risk
Authority Failure	Attribution of authorization	The system cannot reliably determine who authorized an AI action or under what authority it was performed.	Actions occur without clear or valid authorization.
Action Control Failure	Constraint on what can happen	The system does not constrain what actions can be generated or executed, including outputs that function as actions.	Actions are generated or executed without being evaluated and determined to be allowed.
Tool / Protocol Supply Chain Failure	Trust of external capabilities	External tools, plugins, and components are invoked as part of system behavior without sufficient validation or constraint.	External capabilities can execute or shape system behavior without control.
Oversight Saturation	Human decision capacity	Human governance mechanisms cannot meaningfully evaluate system activity at the speed and scale at which it occurs.	Decisions required for control are not actually being made.
Context Integrity Failure	Trust of inputs	The system does not distinguish between trusted and untrusted sources of information, or does not enforce that distinction.	Untrusted or compromised inputs are treated as valid context.

Failure Mode	What Fails	Definition	Core Risk
Verification Collapse	Ability to validate behavior	The system produces behavior faster than it can be meaningfully validated, evidenced, and assured.	System behavior outpaces the ability to establish validation and assurance.
Visibility Failure	Observability of system presence and activity	The organization cannot reliably determine where AI systems are operating or what activities they are performing.	System presence and activity cannot be reliably observed or enumerated.

Taken together, these failure modes describe the problem space of AI security. The important point for future AI failure analysis is that most real-world failures occur within system architecture, after the model layer. They emerge in the gaps between control silos, where model governance, identity, endpoint controls, application security, tooling, and execution are each managed separately, while the agent lifecycle spans all of them.

Admissibility addresses this cross-silo control gap by requiring authority, action, context, and evidence to be evaluated together before execution occurs.

Connection to the Admissibility Pillars

The Admissibility Framework addresses these failure modes directly.

Each pillar maps to a fundamental aspect of system control:

- Authority → identity, ownership, and permission
- Actions → execution boundaries and allowed changes
- Context → trust and influence over reasoning
- Evidence → visibility, traceability, and accountability

Failure Mode	Primary Pillar Control Surface	Notes
Authority Failure	Authority	Breakdown in identity, ownership, or authorization

Failure Mode	Primary Pillar Control Surface	Notes
Action Control Failure	Actions	Failure to constrain what actions can be generated or executed
Context Integrity Failure	Context	Untrusted or unsegmented inputs influence system behavior
Tool / Protocol Supply Chain Failure	Actions	External capabilities are invoked without sufficient constraint or validation
Visibility Failure	Evidence	System presence and activity cannot be reliably observed
Oversight Saturation	Actions	Required decisions cannot be meaningfully evaluated at system scale
Verification Collapse	Evidence	System behavior outpaces the ability to produce verifiable evidence

These failures are distinct, but can interact and compound as systems scale. They arise from how systems are designed and controlled. The solution is to enforce system constraints. Admissibility introduces control cost, but removes unbounded risk.

We've Seen This Problem Before

There are precedents for the kind of thinking proposed here: security models. These focus on defining allowed interactions between subjects (active parties) and objects (passive parties) at any given moment.

In the 1970s and 1980s, formal security models focused on defining the conditions under which system behavior could be considered acceptable were developed. The word "formal"

distinguishes these models from written security policy. A policy might say "employees must not share passwords." A formal model mathematically proves that if all state transitions follow defined rules, then a specific security property (such as "no information flows from high classification to low classification") is provable.

1973 - Bell–LaPadula was the first mathematical model of a multi-level security policy. It was created to formally prove that a computer system handling multiple levels of DoD classified information (Unclassified, Confidential, Secret, Top Secret) would never allow information to flow from a higher classification to a lower one. Before BLP, multi-level security was enforced through physical separation and procedural controls. BLP provided the mathematical foundation for building systems that could securely process information at different classification levels simultaneously.

1977 - The Biba Integrity Model was developed as a complement to Bell-LaPadula. Where BLP focuses on preventing unauthorized information disclosure (confidentiality), Biba focuses on preventing unauthorized information modification (integrity). Biba is often described as the "inverse" or "dual" of Bell-LaPadula because its rules are the mathematical mirror image of BLP's rules.

1983 - The Trusted Computer System Evaluation Criteria (Orange Book) set basic standards for security systems to help the DoD find products that would maintain data confidentiality. It introduced the concept of the Trusted Computing Base (TCB), the portion of the system responsible for enforcing security using the principle that there are some functions that must work correctly for security to be possible and enforceable.

1987 - Clark–Wilson shifted formal control into commercial security requirements from their DoD roots. This model introduced the concept of *well-formed transactions* where transactions are carefully designed and enforced. It requires that *transactions by authorized subjects be evaluated by another party before they are committed* in the system. It arbitrates that e*very interaction between subject and object follows a defined set of rules.*

Seen in this context, the Admissibility Framework can be understood as an extension of earlier control models. It applies the principles of Clark-Wilson and related architectures to systems that include probabilistic reasoning.

All of these models work at the system level to enforce rules that allow or prevent actions. Traditional systems required these in-system control points. Many AI systems do not have them; they generate actions and execute them without an independent decision point. That absence creates unpredictability.

In earlier systems, unpredictability was treated as a critical failure condition. Programming guides warned explicitly: if certain constraints were not enforced, the result would be unpredictable. Unpredictable behavior was not tolerated. It was considered worse than failure.

A system that failed stopped. A system that behaved unpredictably could not be trusted.

Unpredictable = untrusted has not changed. What has changed is that many AI systems now operate without the control required to prevent unpredictability. The result is the same unpredictable conditions traditional systems were specifically designed to avoid.

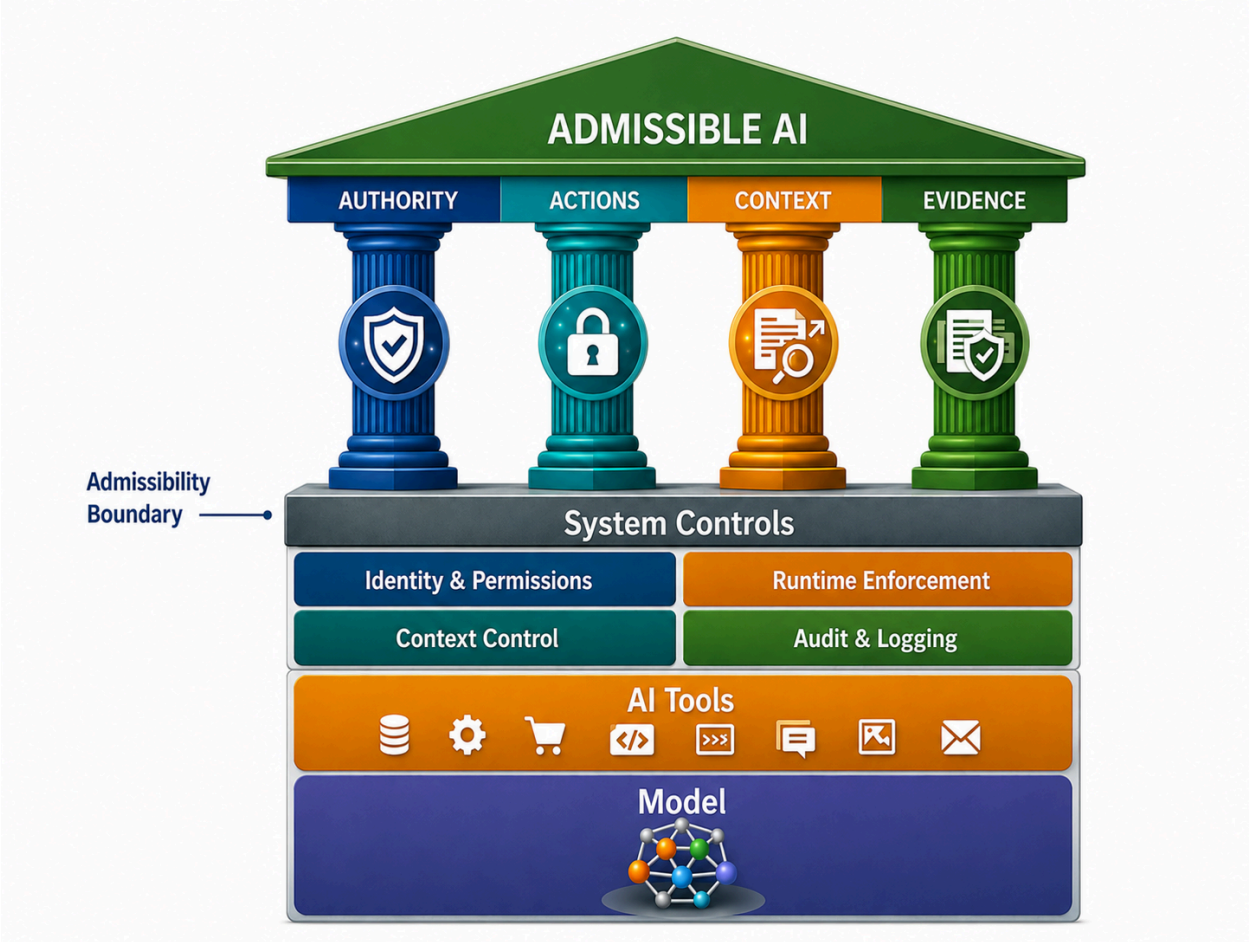
AI systems introduce a non-deterministic reasoning layer between intent and execution. A probabilistic untrusted component now sits between the user and the system action, a distinct break from traditional systems which relied on the core assumption that the program itself can be trusted to behave correctly.

These security models define the conditions under which system behavior is acceptable. Admissibility does the same thing with one critical difference.

Where earlier models assumed deterministic execution, this framework assumes unpredictability and builds enforcement around it.

Control Architecture

The diagram below shows how AI becomes governable in a real system.



Model Layer – The Cognitive Engine

At the base is the AI model. It performs probabilistic reasoning, generates outputs, and interprets inputs. It is capable, but not controllable. It cannot enforce rules or guarantee outcomes. So, the model is treated as an untrusted reasoning component within the system.

AI Tools Layer – Execution Capability

Above the model are tools and integrations. These extend the system from reasoning into action: APIs, databases, repositories, messaging systems, and automation platforms. Once tools are introduced, the system can retrieve sensitive data, execute transactions, and modify production systems. At this point, the system is no longer generating outputs, it is producing effects.

System Controls Layer - Control Mechanisms

Above the tools is the system controls layer. This is where the system provides the mechanisms required to enforce governance decisions.

Four control surfaces operate here:

- **Identity and Permissions** determine who or what the system is acting as.
  Actions execute under defined identities, scoped credentials, and explicit ownership. Non-human actors, delegated authority, service accounts, and tool-specific permissions must be visible, bounded, and revocable.

- **Runtime Enforcement** determines what the system is allowed to do.
  Authorization checks, policy engines, permission gates, and execution boundaries constrain behavior before action occurs. This is where proposed actions are evaluated, approved, denied, or escalated. Runtime enforcement must also preserve execution integrity: the action executed must match the action approved, and intermediaries must not be able to alter executable output outside the control boundary.

- **Context Control** determines what information can influence behavior.
  Inputs, retrieval sources, memory, connector outputs, and environmental state are governed to prevent untrusted or inappropriate influence. Context must be bounded, relevant, and attributable so that the system does not treat arbitrary data as instruction, authority, or valid operating context.

- **Audit and Logging** determine whether behavior can be reconstructed.
  Execution traces, decision records, telemetry, and retained system state ensure actions are observable, attributable, and accountable. Evidence must support reconstruction of the model, prompt, context, tool, policy, configuration, dependency, and execution-state conditions that shaped behavior, so incidents can be investigated and actions evaluated against defined intent.

The critical point is that these control surfaces determine whether proposed behavior is allowed to become action. The system may reason, plan, retrieve, and propose across many layers, but governance becomes enforceable only where authority, action, context, and evidence are resolved before execution.

Admissibility Boundary

In the diagram, the Admissibility Boundary represents the enforcement point where system controls are applied, not a standalone product or component.

The Admissibility Boundary is where proposed actions are evaluated and either permitted or denied before execution occurs. It is not a component of the system, but a control point within it. All behavior generated by the model and assembled through tools must pass through this boundary before it can affect the system.

The system controls provide the mechanisms required to evaluate that behavior:

- identity is resolved
- actions are constrained
- context is validated
- evidence is produced

The Boundary uses those mechanisms to make a single determination: whether the proposed action is allowed. That *determination must also be bound to evidence*. The system must record what action was proposed, which conditions were evaluated, what decision was made, and whether the executed action matched the approved action.

If the required conditions are satisfied, execution proceeds. If they are not, it does not.

The system can generate behavior anywhere, but it can only permit execution here.

Governance Layer – The Pillars

Above the system controls are the governance pillars:

- Authority
- Actions
- Context
- Evidence

These define the conditions that must be satisfied before any action is permitted. They are enforced through the system controls layer below.

Governance Outcome: Admissible AI

These elements form a single control architecture.

- The model generates behavior.
- Tools extend that behavior into action.
- System controls provide the mechanisms required to evaluate it.
- The Admissibility Boundary determines whether it is allowed.
- The pillars define the conditions that decision must satisfy.

Together, they ensure that actions are not executed based on model output alone, but only when required conditions are explicitly evaluated and met.

Without this structure, systems move directly from reasoning to execution. With it, every consequential action is subject to an enforceable decision.

Admissibility in Real Life

In modern systems, execution does not occur in one place. It occurs across APIs, orchestration layers, agents, services, and infrastructure. As a result, the Admissibility Boundary is not implemented as a single centralized mechanism. It is implemented as one or more enforcement points positioned at the moment where an action would affect the system.

These enforcement points already exist in most environments in partial form. API gateways validate requests. Identity systems resolve actors. Policy engines evaluate conditions. Execution services carry out operations. Logging systems record outcomes. The gap is not the absence of controls. It is the absence of a required decision at the point where behavior becomes action.

In practice, the placement and form of an Admissibility Boundary will vary by system. It is determined through threat modeling: where can AI-generated behavior affect the system, and where must a decision be enforced before that effect occurs?

Consider an AI coding assistant operating in an enterprise environment.

Without Admissibility governance:

> User → AI → repository → deployment → production

The AI can push code, modify infrastructure, or expose sensitive data without constraint.

With admissibility applied:

> User → AI → Admissibility Boundary → repository → deployment → production

Before any action is executed, it is evaluated at the boundary.

- Authority is defined through identity, roles, and scoped credentials.
- Actions are limited to approved APIs, repositories, and execution paths that enforce defined conditions.

- Context is restricted to trusted sources and filtered inputs.
- Evidence is captured through telemetry, traceability, and artifact history sufficient to reconstruct system behavior.

If these conditions are satisfied, the action proceeds. If they are not, it does not. The system now determines what is allowed before execution occurs and the AI becomes admissible.

The difference is that controls are required to produce a unified decision before execution occurs.

The Structural Case for Admissibility

Across research and industry guidance, there is growing recognition that AI cannot be governed at the model level alone. Control must exist at the system level.

System-level control is reflected in multiple areas of work. Research increasingly argues that security must be enforced architecturally rather than through model behavior. Industry guidance is focusing on agent autonomy, tool use, and real-time decision-making as the primary sources of risk. Emerging frameworks propose controls across design, runtime, and audit layers to constrain agent behavior.

There is also convergence around key concerns: agent identity, authorization, runtime control, monitoring, and auditability. Proposed architectures introduce policy enforcement layers, privilege separation, and telemetry to address these risks.

The direction is correct and the AI Admissibility structure enables it.

How Admissibility Differs from Existing Frameworks

Despite covering similar problem areas, existing frameworks do not reduce the system to a minimal set of control surfaces.

They do not describe governance in terms of:

- Authority
- Actions
- Context
- Evidence

They also do not organize the threat landscape into a small number of structural failure modes.

Instead, most frameworks are organized around policy categories:

- Ethics and fairness
- Data governance

- Model evaluation
- Security and robustness
- Oversight and compliance

These categories describe important concerns, but they do not map directly to enforcement points in a system. The Admissibility Framework takes a different approach. It does not organize governance by topic, but by control surface. Instead of describing what should be governed, it defines where control must exist based on governance intent.

In some cases, agent governance is grouped into concepts such as alignment, control, visibility, security, and societal integration. These are important concerns. But they are not system control surfaces. They describe what should be addressed, not where control actually exists.

Most frameworks ask: How should AI be governed?

The AI Admissibility Framework asks: Where can AI actually be controlled?

AI Admissibility is a narrower question, but it is also more actionable. It forces governance to align with the architecture of the system.

Traditional security frameworks such as OWASP Top 10, NIST 800-53, and MITRE ATT&CK focus primarily on software vulnerabilities, control requirements, and attack techniques. Emerging AI-focused frameworks, including the OWASP Top 10 for Agentic Applications and related agentic AI security guidance, move closer to the problem by addressing risks such as goal hijacking, tool misuse, identity and privilege abuse, memory and context manipulation, and unexpected execution.

These frameworks are valuable because they identify important classes of AI and agentic risk. But they still primarily describe what can go wrong. The Admissibility Framework addresses a different question: whether the system permits behavior that should not be allowed.

This is a different class of problem: system governance failure. The question is not whether a component is vulnerable, but whether the system allows behavior that should not be allowed, shifting the focus from vulnerabilities to conditions of operation.

Enforceable Governance

Most frameworks describe governance as a set of principles. The Admissibility Framework defines governance as a set of *enforceable conditions*. It does not attempt to make the model safe. It defines the conditions under which the system is allowed to operate.

The result is a governance model that is smaller, more precise, and more directly implementable. It makes governance effective through enforcement at control points. Instead of describing desired behavior, it defines enforceable boundaries. Instead of evaluating the model, it governs the system in which the model operates.

Alignment with Established Security Models

The AI Admissibility structure is closer to established security architecture than to typical AI governance discussions.

It aligns with:

- operating system security models, where access is mediated and enforced
- reference monitor architectures, where access to protected resources is mediated by an independent enforcement point
- transaction integrity models, where actions are constrained by defined rules

These models enforce behavior.

Alignment with Existing AI Frameworks

Many current efforts address similar concerns around AI governance, system behavior, and risk. The distinction is not the existence of those concerns, but where control is located. Most AI frameworks describe categories of risk, governance practices, or lifecycle obligations. The AI Admissibility Framework organizes governance around the control surfaces that must converge before AI-driven behavior is allowed to affect a system.

NIST Cybersecurity Framework

The four Admissibility Pillars align closely with the NIST Cybersecurity Framework, making the model immediately translatable to regulators, auditors, and security leaders.

The CSF is organized around six functions: Govern, Identify, Protect, Detect, Respond, and Recover. These functions describe how organizations manage cybersecurity risk across policy, protection, detection, and response.

The Admissibility Framework operates at a different level. Rather than organizing activities, it *defines the conditions that must be satisfied before system actions are permitted*.

The alignment is as follows:

- **Authority** aligns to **Govern, Identify, and Protect**. It defines who the system is acting as, including identity, ownership, and access control.
- **Actions** align to **Protect**. They define what the system is allowed to do through authorization, least privilege, and execution constraints.
- **Context** aligns to **Protect and Detect**. It governs what information influences behavior and introduces mechanisms to detect when that influence becomes unsafe.
- **Evidence** aligns to **Detect and Respond**. It ensures that actions can be observed, analyzed, and reconstructed through logging, monitoring, and traceability.

This makes the framework directly understandable within an established model.

However, the two approaches differ in structure. The CSF distributes control across functions. The Admissibility Framework resolves control at the single point where actions are evaluated before execution.

Lifecycle and Maturity Models

Lifecycle and maturity models evaluate how well an organization manages AI systems over time. They are useful for assessing program quality, governance processes, organizational capability, and improvement across the AI lifecycle.

They typically ask questions such as:

- Are roles and responsibilities defined?
- Are risks assessed throughout the lifecycle?
- Are models monitored after deployment?
- Are governance practices improving over time?

These are important questions, but they operate at the program and process level.

The AI Admissibility Framework operates at the system-decision level. It does not measure whether governance processes exist or whether organizational maturity is improving. It evaluates whether a specific AI-driven action should be permitted before it affects the system.

A mature AI governance program may still allow unsafe behavior if decisions at the point of action are not constrained. Conversely, an admissibility decision does not replace lifecycle governance; it depends on mature governance to define policy, ownership, risk tolerance, and control intent.

Lifecycle and maturity models ask whether the organization is improving its governance of AI. The AI Admissibility Framework asks whether the system is allowed to act now.

Admissibility supports maturity by making governance operationally testable. It shows whether governance practices are expressed in the system as enforceable conditions, not merely documented as processes.

ISO/IEC 42001: AI Management Systems

ISO/IEC 42001 provides a management system structure for governing AI. It addresses how an organization establishes, implements, maintains, and improves the processes, roles, controls, and risk treatment activities needed for responsible AI use.

The AI Admissibility Framework asks whether the expectations created by that system are actually expressed in the technical environment as enforceable conditions. ISO/IEC 42001 can

define the governance program. Admissibility tests whether that governance reaches the point of action.

A 42001-aligned organization may have policies, accountability structures, risk assessments, and lifecycle controls. Those are necessary, but they do not by themselves determine whether a specific AI-driven action should be allowed to execute.

Admissibility complements ISO/IEC 42001 by asking the operational question:

> Has the system enforced the conditions required for this action to occur?

NIST AI Monitoring and Observability (800-4)

Recent work on AI monitoring reinforces several of the limitations discussed earlier. It acknowledges that AI systems cannot be fully understood or validated prior to deployment, and that behavior must be observed in operation.

Monitoring focuses on tracking, measurement, and evaluation. It detects drift, identifies unexpected behavior, and surfaces outcomes that were not anticipated. It is both necessary and inherently observational.

> Monitoring answers: What did the system do?
> It does not answer: Should the system be allowed to do it?

Monitoring frameworks assume the system is already operating. They focus on what to observe, how to observe it, and how to respond after the fact. They do not define or enforce a decision boundary at the point of action. The distinction between these approaches can be summarized like this:

> Monitoring explains behavior.
> Admissibility governs it.

NIST itself highlights challenges such as lack of visibility, unclear responsibility, rapid system change, and insufficient enforcement mechanisms.

These map directly to the Admissibility model:

- Lack of visibility corresponds to evidence
- Unclear responsibility corresponds to authority
- Dynamic system behavior corresponds to context
- Tool and system complexity correspond to actions
- Lack of enforcement corresponds to Admissibility Boundary

Monitoring is necessary to understand AI systems in operation. But without a mechanism to constrain actions before execution, it becomes a record of behavior rather than a control on it.

The Admissibility Framework addresses that missing layer by defining the conditions under which behavior is allowed to occur at all.

Advantages of the Framework

Admissibility as a Governance Loop

The Admissibility Framework reflects a deeper structure than a set of governance concepts. It forms a complete control loop.

Autonomous systems such as aircraft autopilots, industrial control systems, robotics, and financial trading platforms are governed through closed control loops. These loops follow a consistent pattern:



The Admissibility Pillars map directly to this structure:

- **Authority** defines who is allowed to act
- **Context** defines what information influences the decision
- **Actions** define what the system is allowed to do
- **Evidence** defines how behavior is observed and recorded

At runtime, these conditions are evaluated at the Admissibility Boundary before execution occurs. The decision record is produced inline as part of enforcement. Once execution occurs, outcome evidence is added and fed back into governance decisions, shaping future authority, policy, and control.

There is a closed governance loop in which:

- behavior is constrained before execution
- outcomes are observed after execution
- governance evolves based on evidence

Why This Matters for AI

Traditional systems followed a linear model: Input → Process → Output, with governance focused primarily on the human who initiated, approved, or was accountable for the action.

AI systems change that structure. The process becomes: Prompt → AI Reasoning → Proposed Action → Admissibility Decision → Execution → Evidence → Governance Update.

The decision-making function now sits inside the system, and system behavior may change based on context, tools, retrieved information, prior steps, and environmental conditions. As a result, governance must also move into the system. It must evaluate authority, context, and actions before execution occurs, produce evidence of the decision, and use that evidence to update future governance.

Many AI governance approaches focus on only part of this loop.

Model safety work emphasizes context.
Security frameworks often focus on actions.
Compliance frameworks emphasize evidence.

The Admissibility Framework includes Authority, Actions, Context, and Evidence, and requires them to converge at the decision point before execution occurs.

Incident and Failure Investigation

The framework also aligns directly with how real-world incidents are investigated.

Across cybersecurity, aviation, financial systems, and industrial environments, investigators follow a consistent sequence of questions.

First: Who initiated the action? This corresponds to Authority.

Second: What did the system do? This corresponds to Actions.

Third: What inputs or conditions influenced the behavior? This corresponds to Context.

Fourth: What evidence confirms the sequence of events? This corresponds to Evidence.

If any of these elements are missing, the investigation fails.

- Without identity, attribution collapses.
- Without action records, impact is unclear.
- Without context, the root cause cannot be determined.
- Without evidence, nothing can be proven.

The framework therefore defines not only how systems are governed, but whether failures can be understood at all.

Mapping to Real Incidents

When applied to real-world events, the seven failure modes consistently align with observed AI incidents.

- Authority failures appear in credential misuse and delegated identity
- Action failures appear in uncontrolled execution of system changes
- Context failures appear in prompt injection and poisoned inputs
- Supply chain failures appear in compromised tools and dependencies
- Oversight failures appear in systems operating beyond human review capacity
- Verification failures appear in unreviewed outputs entering production
- Visibility failures appear in unmanaged or unknown AI deployments

These are not failures of model behavior. They are failures of system design.

Alignment with Modern System Architecture

The framework is inherently aligned with modern system architecture.

Contemporary cloud-native and distributed systems are typically designed around a separation of concerns, often described as a control plane and data plane model, where identity, execution, data access, and observability are handled by distinct components.

In these architectures:

- **Identity systems** define who can act
- **Execution systems** define what can occur
- **Data access controls** define what information can influence behavior
- **Observability systems** define what can be seen, monitored, and reconstructed

Regulatory Clarity

The framework also provides a simple model for governance assurance.

Autonomous systems can be considered governable if four questions can be answered:

- Who authorized the system to act?

- What actions could it take?
- What information influenced its decisions?
- Can the outcome be reconstructed?

Answering these is sufficient to make the system both controllable and investigable.

A Minimal Model

The structure is deliberately minimal:

- Four control surfaces.
- Seven failure modes.
- System control mechanisms.

Models with this level of simplicity tend to emerge only after a field matures. Examples include the CIA triad, the OSI model, Zero Trust, and the CAP theorem.

It is not about simplifying the problem, it is about revealing its essence. The reduction to minimal structure suggests the AI Admissibility Framework is not introducing a new idea. It is making an existing pattern visible and actionable in AI systems.

The Cost of Bounded Behavior

AI governance discussions often assume that lower friction is inherently better. Faster deployment, smoother user experience, fewer approval steps, and reduced operational overhead are treated as signs of maturity. In many cases, reducing friction is valuable. But in AI-enabled systems, especially agentic or tool-using systems, some friction is not administrative waste. It is the mechanism by which system behavior becomes bounded.

Threat modeling, runtime admissibility checks, authority resolution, context validation, evidence capture, and action gating all introduce cost. They require design effort and maintenance. They may add architectural complexity. In some cases, they may introduce latency. These concerns are real and should not be dismissed.

But removing these mechanisms does not remove risk. It removes the system's ability to constrain, explain, and reconstruct behavior before harm occurs.

A system can be optimized for speed, adoption, and user convenience while remaining structurally ungovernable. If an AI system can act without explicit authority, use unvalidated context, invoke tools without bounded permissions, or leave no evidence trail sufficient for reconstruction, then its efficiency has been achieved by transferring risk into the operating environment.

The practical tradeoff is not friction versus no friction. It is *designed friction versus unbounded and potentially silent failure.*

AI adoption can create the appearance of progress when organizations treat usage, deployment, or labor substitution as evidence of success without determining whether the resulting system is governable.

The risk is not AI adoption itself. The risk is that organizations remove human judgment, review, and process friction before understanding which parts of that friction were waste and which parts were functioning as informal controls. A human-in-the-loop workflow may be slow, but it often contains embedded forms of contextual judgment, escalation, skepticism, exception handling, and accountability. If those functions are removed without being replaced by explicit authority boundaries, action constraints, context controls, and evidence requirements, the organization has changed the risk structure of the system by replacing informal human control with ungoverned system behavior.

Some human effort is waste. Some of it is control. Some of it is judgment. Some of it is evidence. When organizations automate away those functions without intentionally replacing them, they may not eliminate cost; they may defer it. The cost returns later as incident response, rework, customer harm, regulatory exposure, brittle operations, or the need to reintroduce human expertise after failure.

Admissibility does not require every action to be slow, manual, or bureaucratic. The goal is not to insert human review everywhere or preserve human involvement for its own sake. The goal is to determine which decisions require enforceable boundaries, which actions can be safely automated, which context must be trusted or constrained, and what evidence must exist after the fact.

Well-designed admissibility should reduce unnecessary governance friction by making the control decision explicit, repeatable, and system-executed.

The question for architects is therefore, “Which forms of failure are we willing to allow the system to produce without prior constraint, timely detection, or reliable evidence?”

Where silent or unconstrained failure is unacceptable, admissibility is not optional overhead. It is part of the system design.

Organizations can choose not to implement admissibility boundaries. But that choice should be explicit. It means accepting that certain AI-enabled failures may occur without prior constraint, without timely detection, and without sufficient evidence to reconstruct what happened.

Authority, Actions, Context, and Evidence are the places where friction must be deliberately designed, minimized where possible, and preserved where necessary.

The goal is not more friction. The goal is the right friction in the right place, enforced by the system rather than improvised after the fact.

The Final Insight

AI systems do not fail because models behave incorrectly. They fail because actions are allowed to occur without an enforced decision.

This paper has shown that the problem is not the model. It is the absence of a system-level control structure that determines whether behavior is permitted before it becomes action. The Admissibility Framework provides that structure.

It defines:

- the conditions required for control
- the point at which those conditions must be evaluated
- and the mechanism by which execution is allowed or denied

The paper does not describe a new set of principles, but the application of established system security concepts to a new class of systems. When these conditions are enforced at the Admissibility Boundary, AI behavior becomes governable.

Actions are no longer derived solely from model output. They are allowed only when they satisfy defined constraints.

Following these principles moves AI governance from influencing behavior to enforcing decisions, and from observing outcomes to controlling execution. The result is a system that can act, but only within defined and enforceable limits. An AI system is safe not when it behaves correctly, but when it cannot act incorrectly.

As AI systems become more autonomous, governance cannot remain descriptive. It must be enforceable, explicit, and applied at the moment of action.

Appendix: Convergence and Emergence

The Admissibility Framework appears simple in its final form. That is not because the problem is simple, but because the same structure emerges repeatedly when systems are examined under real operating conditions. The pattern becomes visible when analyzing how systems fail.

Across a wide range of problems, including agent architectures, prompt injection, retrieval poisoning, tool supply chains, execution risk, runtime permissions, observability gaps, and governance models that do not align to system behavior, the same underlying question arises: How do we determine whether a system should be allowed to act?

The answer resolves into a small set of governing conditions:

- Authority
- Actions

- Context
- Evidence

These conditions reduce complexity to a form that can be evaluated and enforced.

Two Problems, One Structure

The framework reflects the convergence of two problems that are often treated separately. The first is technical: How do systems actually fail? The second is conceptual: How are those failures described in governance terms? Most approaches address one or the other. The AI Admissibility Framework connects them.

Each pillar corresponds to a class of system failure:

- Authority → identity breakdown and attribution failure
- Actions → uncontrolled or unauthorized execution
- Context → untrusted or unsafe influence
- Evidence → inability to observe, reconstruct, or explain behavior

The alignment allows technical failure and governance language to describe the same structure.

Convergence Across Domains

The structure does not originate from a single domain. It appears across:

- established computer security models
- cloud-native system architectures
- cybersecurity frameworks such as NIST Cybersecurity Framework
- incident investigation and forensics practices
- risk and governance frameworks
- autonomous system control loops

When a structure emerges consistently across domains, it typically reflects underlying constraints rather than design choices. For those constraints to be realized, they must be expressed, evaluated, and continuously informed by real-world conditions if they are to govern real-world systems.

Concept, Mechanism, and Signal

The AI Admissibility work is supported by three aligned elements:

- A conceptual model that defines the governing conditions: the AI Admissibility Framework
- A mechanism for evaluating those conditions in real systems: the Control Intent Engine
- A continuous intake of signals that reflect emerging risks and failure patterns: the AI governance “Doctor’s Bag”

These are presented as aligned elements of a single governance approach. Together, they reinforce one another.

The model defines what must be evaluated. The mechanism enables that evaluation. The signals ensure the model remains grounded in real-world system behavior.

Why the Framework Is Emerging Now

The field is currently fragmented.

- AI safety research focuses on model behavior and alignment
- Security engineering focuses on attack surfaces and vulnerabilities
- Governance and policy focus on oversight, ethics, and regulation

These perspectives address different aspects of the problem, but do not provide a unified structure for governing system behavior at runtime. The Admissibility Framework operates at that level.

Where Admissibility Becomes Necessary

Admissibility is not required in all systems. It becomes necessary when systems reach a level of autonomy and complexity that cannot be governed through existing approaches.

This point is reached when:

- systems act beyond their originally defined scope
- behavior cannot be reliably predicted or constrained
- actions produce effects that must be evaluated before execution
- incidents cannot be fully reconstructed after the fact
- governance mechanisms fail to prevent undesirable outcomes

At that point, governance can no longer rely on policy, behavior, or after-the-fact response. The system must determine, in real time, whether actions are allowed. Admissibility provides the structure that enables that determination.

The Result for Governance

AI governance, as currently practiced, is not producing reliable control because it is not applied at the point where control can be established. When systems are capable of acting, governance cannot remain descriptive. It must be enforced.

The structures that enable enforcement are not new. They include clear conditions, executable evaluation, and continuous signal. They appear wherever complex systems must be controlled and understood. The Admissibility Framework makes these structures explicit for AI systems.